

GUIA DO DESENVOLVEDOR

VulnScan Toolkit

Configuração, Estrutura do Projeto e Guia de Contribuição

Versão 0.1 (MVP) · Julho de 2026

Autor: Higor Silva

1. Pré-requisitos

- Docker & Docker Compose (forma recomendada de rodar e desenvolver contra a ferramenta)
- Python 3.11+ (necessário apenas para desenvolvimento local, fora de container)
- Git

Desenvolva e teste sempre contra um alvo de laboratório/autorizado (ex: DVWA, OWASP Juice Shop, WebGoat rodando localmente). Nunca aponte a ferramenta para um alvo sem autorização explícita — o config/scope.yaml bloqueará de qualquer forma.

2. Primeiros Passos

```
git clone <repo-url> vulnscan-toolkit
cd vulnscan-toolkit
docker compose build
docker compose run --rm vulnscan --help
```

Para desenvolvimento local (sem Docker), instale as dependências diretamente e garanta que sqlmap, Nikto, Nuclei e o script baseline do ZAP estejam disponíveis no PATH :

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cd src && python main.py --help
```

3. Estrutura do Projeto

```
vulnscan-toolkit/
├── Dockerfile           # Imagem do container: instala Python + todas as ferramentas
                          # externas
├── docker-compose.yml   # Wrapper de build/run, monta ./reports e ./config
├── requirements.txt
├── config/
│   ├── scope.yaml       # Allowlist de alvos (edite antes de varrer qualquer coisa)
│   └── settings.yaml    # Timeouts e parâmetros ajustáveis
├── reports/             # Saída dos scans, uma subpasta por run_id
├── src/
│   ├── main.py          # Ponto de entrada da CLI
│   ├── core/
│   │   ├── finding.py   # Modelo Finding + Severity (o contrato módulo <->
relatório)
│   │   ├── scope_guard.py # Verificação da allowlist
│   │   └── orchestrator.py # Resolve e executa os módulos
└── modules/
```

```
|   └─ base.py          # Classe abstrata BaseScanModule
|   └─ registry.py      # MODULE_REGISTRY + PROFILES
|   └─ recon.py, sqli.py, xss.py, xxe.py
└─ report/
    └─ generator.py      # Agregação + renderização
    └─ templates/report.html.j2
```

4. Conceitos Centrais

4.1 O contrato Finding

Todo módulo de varredura deve retornar uma `list[Finding]` (definida em `core/finding.py`). Esta é a única interface entre um módulo e o motor de relatórios — nunca contorne esse contrato escrevendo diretamente no relatório ou retornando a saída bruta da ferramenta.

```
Finding(
    module="xss", title="...", description="...",
    severity=Severity.HIGH, cvss_vector="...", cvss_score=7.1,
    endpoint="https://alvo/app", parameter="q",
    evidence="...", recommendation="...",
    references=[...], tool_source="owasp-zap",
)
```

4.2 BaseScanModule

Todo módulo herda de `modules/base.py::BaseScanModule` e implementa um único método: `run(self) -> list[Finding]`. A classe base fornece `self._run_cli(cmd, timeout)`, um helper que executa um comando CLI externo via `subprocess` e persiste seu `stdout/stderr` em `self.work_dir` para inspeção posterior.

5. Adicionando um Novo Módulo de Varredura

Para integrar uma nova ferramenta ou classe de vulnerabilidade (ex: um checker de CSRF, ou um módulo de SSRF usando outra ferramenta):

1. Crie `src/modules/<nome>.py` com uma classe herdando de `BaseScanModule`.
2. Implemente `run()`: monte o comando CLI, chame `self._run_cli(...)`, faça o parsing da saída da ferramenta (JSON/JSONL/regex sobre `stdout` — o que a ferramenta oferecer) e retorne uma lista de objetos `Finding`.
3. Registre a classe em `src/modules/registry.py` dentro de `MODULE_REGISTRY`, e adicione-a às entradas relevantes de `PROFILES` caso deva rodar por padrão em `quick / full`.
4. Se a ferramenta precisar ser instalada no container, adicione o passo de instalação ao `Dockerfile`.

5. Adicione um teste unitário para o parser em `tests/`, alimentando-o com uma amostra salva da saída bruta da ferramenta.

Nenhuma alteração em `orchestrator.py` ou `report/generator.py` deveria ser necessária para adicionar um módulo — se você se pegar editando esses arquivos só para plugar uma nova ferramenta, o módulo não está seguindo o contrato corretamente.

6. Convenções de Código

- Type hints em todas as funções/métodos públicos; use `from __future__ import annotations` para referências futuras.
- Módulos nunca devem lançar exceções não tratadas que interrompam toda a execução — o orquestrador já encapsula cada chamada de módulo em `try/except` e registra falhas em log, mas os módulos devem tratar os modos de falha esperados da ferramenta (timeouts, arquivos de saída ausentes) de forma resiliente, retornando uma lista vazia em vez de propagar o erro.
- Nunca escreva diretamente nos arquivos de `stdout/stderr` de `self.work_dir` exceto através de `_run_cli` — mantenha a saída bruta das ferramentas centralizada para fins de auditoria.
- Mantenha a lógica de parsing (regex, suposições de schema JSON) isolada no arquivo do módulo, sem vaziar para `core/`.

7. Testes

Os testes unitários ficam em `tests/` e usam `pytest`. Priorize testar a lógica de parsing de cada módulo contra amostras de saída salvas (fixtures), em vez de rodar as ferramentas externas reais em CI, já que isso exigiria acesso de rede a um alvo real e seria lento/não-determinístico.

```
pip install pytest
pytest tests/ -v
```

Para validação ponta a ponta, rode o pipeline completo contra uma aplicação vulnerável local (DVWA, Juice Shop, WebGoat) subida em um container/rede separado, e inspecione o relatório gerado quanto à correção.

8. Estendendo o Template do Relatório

O template do relatório (`report/templates/report.html.j2`) é Jinja2 puro + CSS inline, renderizado de forma idêntica tanto para a saída HTML quanto PDF (PDF via WeasyPrint sobre a mesma string HTML). Para adicionar uma nova seção, estenda o template e, se necessário, adicione campos agregados de suporte em `ReportGenerator._build_summary()`. Mantenha o CSS compatível com impressão (evite recursos CSS3 não suportados — o WeasyPrint tem suporte parcial a CSS) e teste a saída em PDF especificamente, já que a renderização HTML em um navegador pode esconder problemas de layout que o WeasyPrint vai expor.

9. Referência da CLI

Flag	Descrição
<code>--target</code>	URL base do alvo (obrigatório)
<code>--profile</code>	<code>quick</code> / <code>full</code> / <code>custom</code>
<code>--modules</code>	Lista explícita de módulos, sobrepõe o profile
<code>--out</code>	Diretório de saída dos relatórios (padrão <code>./reports</code>)
<code>--format</code>	<code>html</code> / <code>pdf</code> / <code>both</code>
<code>--scope-file</code>	Caminho para o arquivo de allowlist
<code>--i-have-authorization</code>	Flag de confirmação obrigatória; a ferramenta recusa rodar sem ela